

Actividad: Benchmark

Luis Fernando Salazar Hernández – A01738527

Instituto Tecnológico y de Estudios Superiores de Monterrey

Diseño de sistemas embebidos avanzados

Emmanuel Torres Ríos

30 de septiembre de 2026

Índice

1. Introducción..... 3
2. Objetivo..... 3
3. Descripción del programa..... 3
3.1. Código utilizado..... 3
4. Modificación de la función principal..... 4
5. Entorno de compilación..... 4
5.1. macOS con Apple Clang..... 4
5.2. Windows con MinGW w64 y GCC..... 5
6. Equipos utilizados..... 5
7. Metodología..... 6
8. Resultados..... 6
9. Análisis de la MacBook Air M1..... 7
10. Análisis de la ASUS Vivobook Pro 15..... 9
11. Análisis de la ASUS TUF F16..... 10
12. Comparación entre equipos..... 11
13. Sobresuscripción de hilos..... 11
14. Consideraciones del benchmark..... 12
15. Conclusiones..... 12

1. Introducción

En esta actividad realicé un benchmark para observar cómo cambia el tiempo de ejecución de un programa paralelo al modificar la cantidad de hilos. Utilicé mi MacBook Air con procesador Apple M1 y comparé mis resultados con los de una ASUS Vivobook Pro 15 y una ASUS TUF F16. El programa está escrito en C y utiliza OpenMP para aproximar el valor de π mediante integración numérica.

2. Objetivo

Analizar cómo cambia el tiempo de ejecución de un mismo programa al utilizar diferentes cantidades de hilos en tres equipos. A partir de los resultados, identificar los mejores tiempos registrados y observar cuándo aumentar los hilos deja de mejorar el rendimiento.

3. Descripción del programa

El programa aproxima π mediante integración numérica. Divide el intervalo de 0 a 1 en 1,500,000,000 pasos, evalúa la función $4/(1 + x^2)$ y suma sus contribuciones. Al multiplicar el resultado por el tamaño del paso obtiene una aproximación que, en la salida documentada, aparece como 3.141593.

OpenMP reparte las operaciones entre los hilos. Cada uno calcula una parte y guarda su suma; después, el programa reúne los resultados. La cantidad de operaciones se mantiene y NUM_THREADS se modifica entre pruebas para cambiar los hilos solicitados.

El tiempo se calcula con omp_get_wtime(), tomando una lectura antes del trabajo y otra después de sumar e imprimir los resultados de cada hilo. Por eso, el tiempo reportado incluye también esa impresión. Las sumas individuales todavía no se han multiplicado por el tamaño del paso, lo que explica que sean grandes sin indicar un error.

3.1. Código utilizado

El código utilizado fue el siguiente. Conserva NUM_THREADS en 2100; durante las pruebas cambié esa constante según la cantidad de hilos que quería evaluar.

```c
#include <stdio.h>
#include <omp.h>
static long num_pasos=1500000000;
double paso;
#define NUM_THREADS 2100
int main(){
 int i, nthreads;
 double pi, sum[NUM_THREADS], t1, t2, tiempo;
 paso=1.0/num_pasos;
 omp_set_num_threads(NUM_THREADS);
```

```
 t1 = omp_get_wtime();
#pragma omp parallel
{
  int i, id, nthrds;
  double x;
  id = omp_get_thread_num();
  nthrds = omp_get_num_threads();
  if (id==0) nthreads=nthrds;

  for (i=id, sum[id]=0.0;i<num_pasos;i=i+nthrds){
    x=(i+0.5)*paso;
    sum[id]+=4.0/(1.0+x*x);
  }
 }
 for (i=0,pi=0.0;i<nthreads;i++){
   pi+=sum[i]*paso;
   printf("(%lf) + hilo (%d)\n", sum[i] , i);
 }
   t2=omp_get_wtime();
   tiempo=t2-t1;
   printf("pi = (%lf)\n",pi);
   printf("tomo (%lf) segundos\n", tiempo);
   return 0;
}
```

4. Modificación de la función principal

Al compilar en macOS, Apple Clang mostró una advertencia porque el código original utilizaba void main(). Para corregirla cambié la declaración a int main() y agregué return 0; al final, como se observa en el código de la sección anterior.

Esta modificación permite que el programa termine con un código de salida correcto y no cambia el cálculo de π. El otro ajuste realizado entre pruebas fue cambiar NUM_THREADS para establecer la cantidad de hilos.

5. Entorno de compilación

5.1. macOS con Apple Clang

En la MacBook utilicé Apple Clang versión 21.0.0 (clang-2100.3.34.2), con destino arm64-apple-darwin27.0.0, y la biblioteca libomp en las rutas de Homebrew. Verifiqué la versión desde la terminal con clang --version. El comando que ocupé para compilar fue el siguiente:

clang -Xclang -fopenmp -I/opt/homebrew/opt/libomp/include
-L/opt/homebrew/opt/libomp/lib -lomp integer_parallel.c -o integer_parallel

El comando se introduce en una sola línea. Las rutas indican dónde están los encabezados y la biblioteca libomp; -o integer_parallel asigna el nombre al ejecutable. Después de compilar, lo ejecuté con:

./integer_parallel

5.2. Windows con MinGW w64 y GCC

En los equipos Windows se utilizó GCC mediante MinGW-w64. La opción -fopenmp habilita el soporte de OpenMP. La sintaxis de compilación es:

gcc -fopenmp integer_parallel.c -o integer_parallel.exe

En PowerShell, el ejecutable se puede invocar desde su directorio con la siguiente llamada:

.\integer_parallel.exe

6. Equipos utilizados

La siguiente tabla reúne las características de los tres equipos y los nombres de sus propietarios. Las frecuencias indicadas son valores máximos de referencia.

Característica	Equipo 1 Luis Fernando MacBook Air M1	Equipo 2 Fátima Vivobook Pro 15	Equipo 3 Magda TUF F16
Procesador	Apple M1	Intel Core Ultra 7 155H	Intel Core i5-13450HX
Arquitectura	ARM64	x86-64	x86-64
Núcleos físicos	8 (4 de rendimiento y 4 de eficiencia)	16 (6 P, 8 E y 2 LP E)	10 (6 P y 4 E)
Hilos lógicos	8	22	16
Frecuencia máxima	3.20 GHz en rendimiento; 2.06 GHz en eficiencia	Hasta 4.80 GHz	Hasta 4.60 GHz
Memoria RAM	8 GB LPDDR4X unificada	24 GB DDR5 a 5600 MT/s	16 GB DDR5
Sistema operativo	macOS	Windows 11	Windows 11
Compilación	Apple Clang y libomp	MinGW-w64 GCC	MinGW-w64 GCC

Los equipos cuentan con distintas combinaciones de núcleos de rendimiento y eficiencia. También cambia la cantidad de hilos lógicos disponibles: 8 en la MacBook, 22 en la Vivobook y 16 en la TUF. Estos recursos permiten interpretar por qué solicitar cientos o miles de hilos no equivale a ejecutarlos todos al mismo tiempo.

7. Metodología

En cada prueba se cambió NUM_THREADS, se compiló el programa con OpenMP, se ejecutó desde la terminal y se registró el tiempo mostrado. La tabla reúne 20 configuraciones, de 4 a 16,000 hilos solicitados, aunque algunas no tienen medición en todos los equipos.

En mi MacBook completé las mediciones hasta 2,000 hilos. Al solicitar 2,100, OpenMP no pudo asignar los recursos necesarios para crear los hilos y el programa se detuvo. El mensaje exacto y su significado se presentan en el análisis de la MacBook. Por eso no continué con cantidades mayores ni registré tiempos posteriores a 2,000 hilos.

En mi caso procuré realizar las pruebas con la mayoría de las aplicaciones cerradas, aunque todavía había procesos en segundo plano. Mis colegas pudieron haber ejecutado el programa mientras tenían otras aplicaciones o procesos activos. Por ello, las condiciones de uso de los equipos pudieron ser diferentes.

8. Resultados

Los tiempos están expresados en segundos; un valor menor indica una ejecución más corta. N/D significa que no se dispone de una medición y no equivale a cero.

Hilos solicitados	Luis Fernando MacBook M1 (s)	Fátima Vivobook (s)	Magda TUF F16 (s)
4	1.484984	2.617	5.522
10	1.479214	6.241	5.984
15	1.478608	1.663	4.500
20	1.427890	1.476	1.174
25	1.384869	1.238	2.710
30	1.364921	2.413	4.150
35	1.370269	3.003	1.437
100	1.168298	0.850	N/D
200	1.155296	0.785	N/D
350	1.100398	0.748	0.738
450	1.076189	0.715	0.703
550	1.099836	0.679	0.631
1,000	1.139168	0.558	0.492
2,000	1.240836	0.526	0.600
10,000	N/D	0.887	0.518
12,000	N/D	0.952	0.952

Hilos solicitados	Luis Fernando MacBook M1 (s)	Fátima Vivobook (s)	Magda TUF F16 (s)
13,000	N/D	1.001	0.952
14,000	N/D	1.071	0.930
15,000	N/D	1.086	0.974
16,000	N/D	1.123	1.047

En la MacBook, los valores N/D de 10,000 a 16,000 hilos corresponden al barrido que no se pudo continuar. En la TUF, las pruebas de 100 y 200 hilos no tienen un tiempo registrado. Por ello, esos puntos no se usan para comparar los equipos.

9. Análisis de la MacBook Air M1

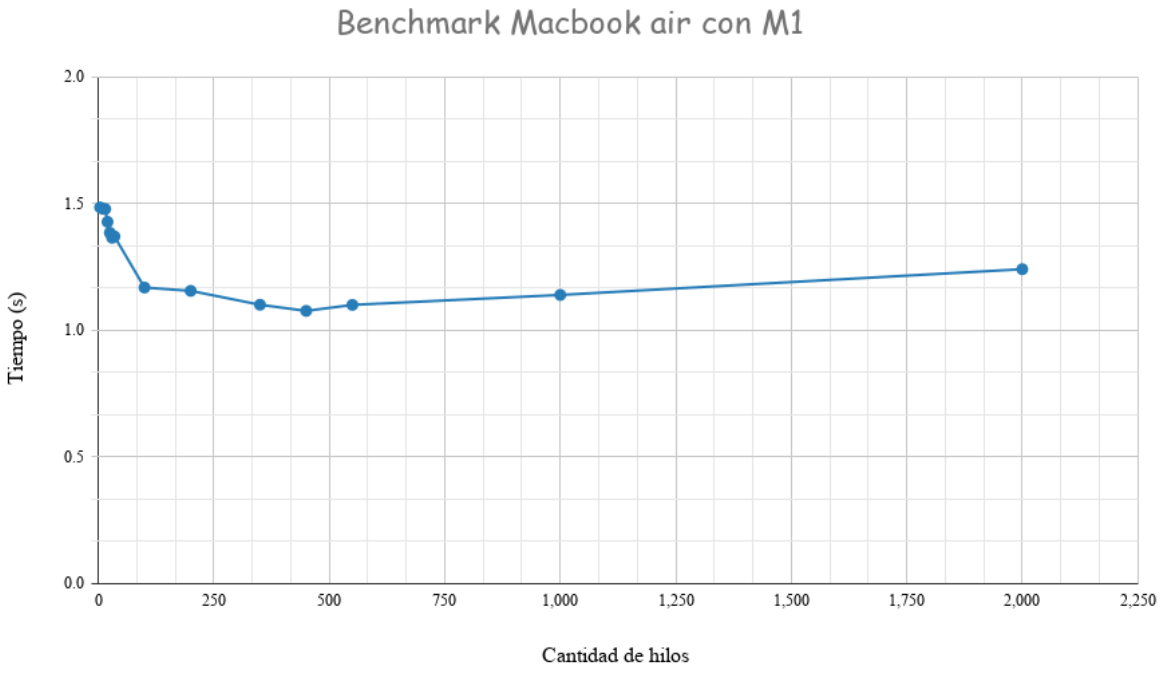


Figura 1. Tiempo registrado en la MacBook Air M1 para cada cantidad de hilos.

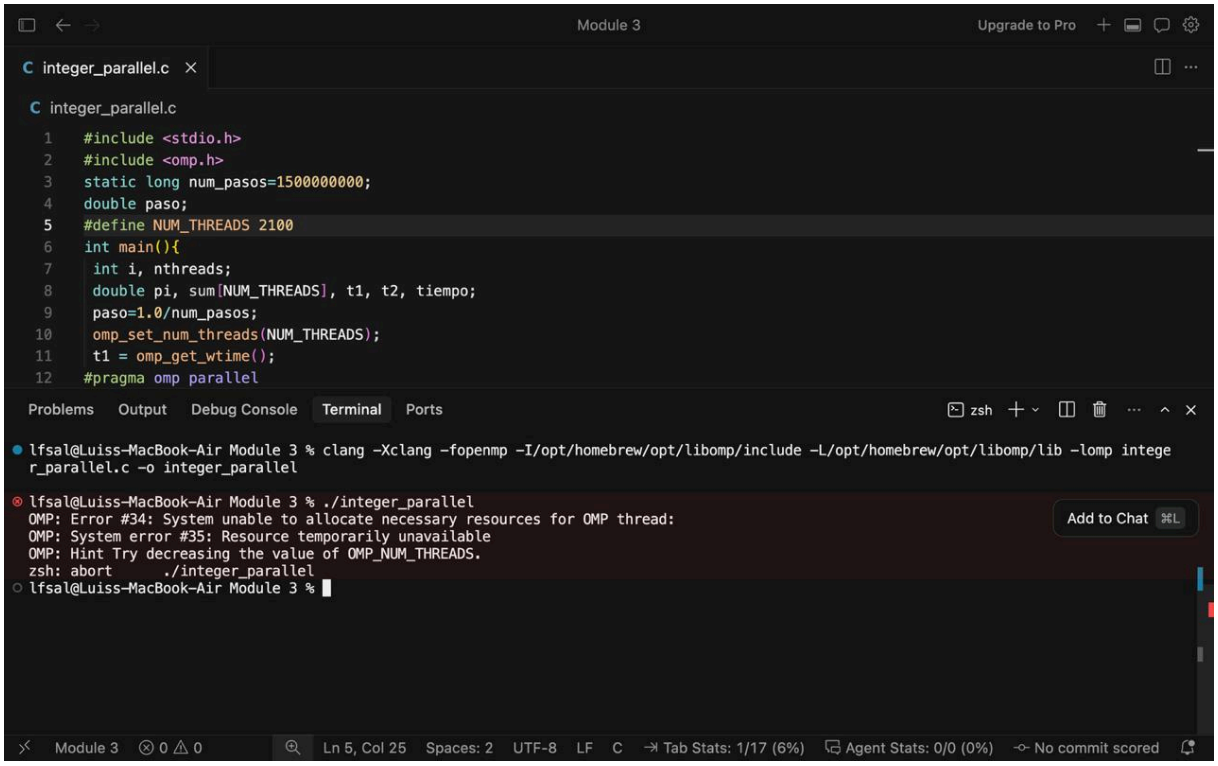
En mi MacBook el tiempo disminuyó de forma relativamente gradual: pasó de 1.484984 segundos con 4 hilos a 1.427890 con 20 y a 1.168298 con 100. El mejor valor registrado fue 1.076189 segundos con 450 hilos. Frente a las curvas de Windows, los cambios entre las configuraciones fueron menos pronunciados.

Después del mínimo, agregar hilos dejó de mejorar el tiempo: con 550 hilos obtuve 1.099836 segundos, con 1,000, 1.139168. La curva muestra una disminución inicial y un aumento posterior, por lo que el mejor resultado de esta serie no se obtuvo con la mayor cantidad de hilos.

Al intentar ejecutar el programa con 2,100 hilos, la terminal mostró exactamente lo siguiente:

OMP: Error #34: System unable to allocate necessary resources for OMP thread.
OMP: System error #35: Resource temporarily unavailable
OMP: Hint Try decreasing the value of OMP_NUM_THREADS.
zsh: abort ./integer_parallel

El mensaje indica que OpenMP no pudo asignar los recursos necesarios para crear los hilos solicitados porque esos recursos no estaban disponibles en ese momento. Por eso sugiere disminuir la cantidad de hilos; la línea "zsh: abort" confirma que el programa se interrumpió. En consecuencia, no obtuve un tiempo con 2,100 hilos y no continué las pruebas después de los 2,000 que sí habían funcionado.



Evidencia del error en la MacBook al solicitar 2,100 hilos.

10. Análisis de la ASUS Vivobook Pro 15

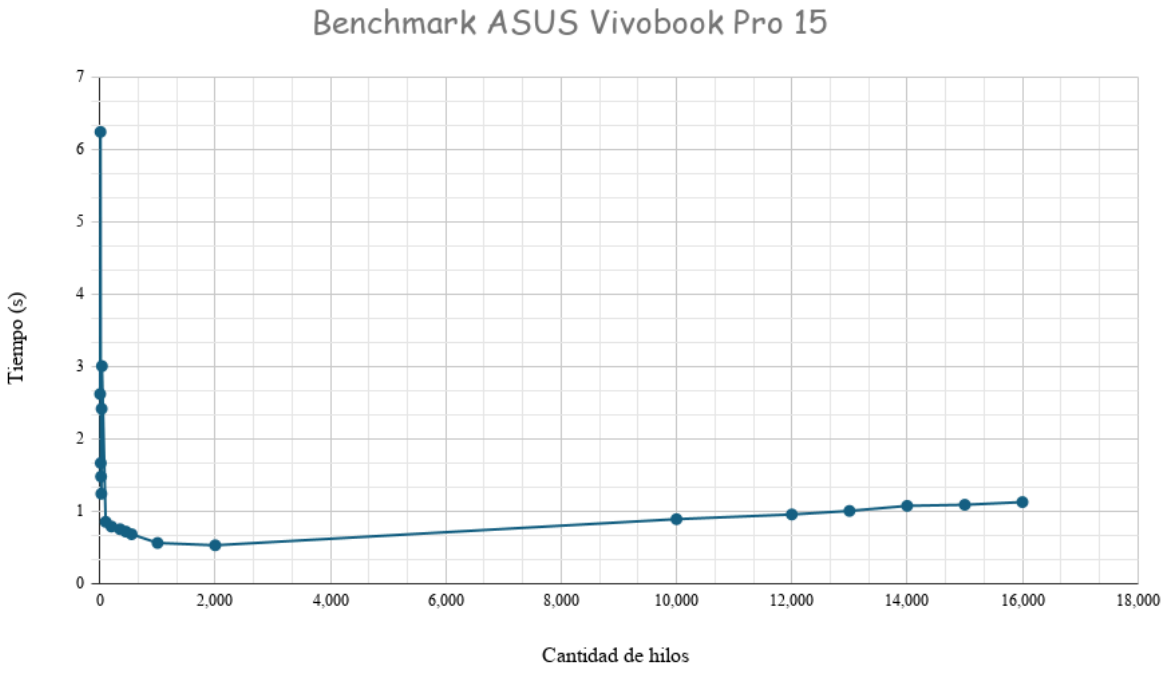


Figura 2. Tiempo registrado en la ASUS Vivobook Pro 15 para cada cantidad de hilos.

La Vivobook presentó variaciones grandes al inicio: registró 2.617 segundos con 4 hilos y 6.241 con 10. Después bajó a 1.663 con 15 y a 1.238 con 25, pero volvió a aumentar con 30 y 35. Por lo tanto, en ese tramo no hubo una mejora constante al agregar hilos.

A partir de 100 hilos, los puntos registrados disminuyeron: 0.850 segundos con 100, 0.748 con 350, 0.679 con 550 y 0.558 con 1,000. El mínimo fue 0.526 segundos con 2,000 hilos, una diferencia considerable frente a las primeras configuraciones.

Al continuar aumentando los hilos, el tiempo subió a 0.887 segundos con 10,000 y llegó a 1.123 con 16,000. Este último valor es más del doble del mínimo. La curva muestra que la mejora tuvo un límite dentro del barrido realizado.

11. Análisis de la ASUS TUF F16

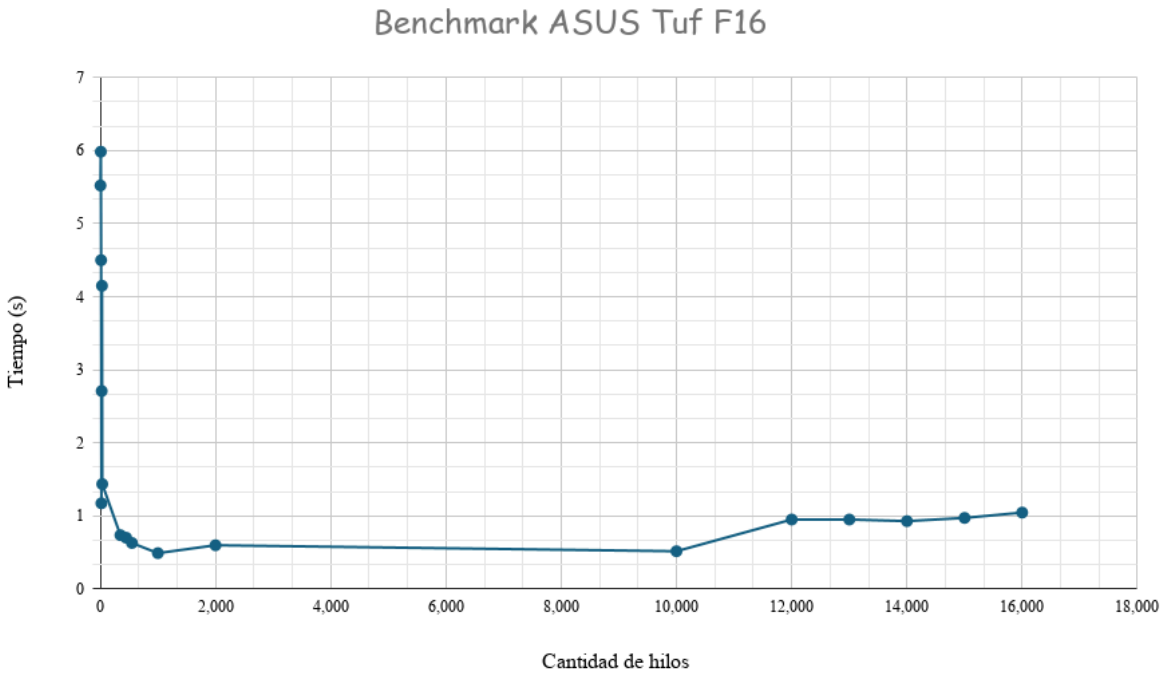


Figura 3. Tiempo registrado en la ASUS TUF F16 para cada cantidad de hilos.

La TUF también mostró variaciones al inicio. Con 4 hilos registró 5.522 segundos y con 10, 5.984. Con 20 bajó a 1.174, pero con 25 y 30 volvió a subir a 2.710 y 4.150. El cambio de tiempo tampoco siguió una disminución uniforme.

Entre 350 y 1,000 hilos, los tiempos bajaron de 0.738 a 0.703, 0.631 y finalmente 0.492 segundos. El valor con 1,000 hilos fue el menor registrado en los tres equipos. No hay datos con 100 y 200 hilos, por lo que no se interpreta ese tramo.

Después del mínimo, el tiempo aumentó a 0.600 segundos con 2,000 hilos y volvió a bajar a 0.518 con 10,000. En las configuraciones siguientes se acercó a un segundo y terminó en 1.047 con 16,000. Aunque hubo fluctuaciones, ninguna de esas mediciones mejoró el resultado de 1,000 hilos.

12. Comparación entre equipos

Equipo y propietario	Hilos	Menor tiempo (s)
Luis Fernando MacBook Air M1	450	1.076189
Fátima ASUS Vivobook Pro 15	2,000	0.526
Magda ASUS TUF F16	1,000	0.492

La MacBook fue más rápida con 4, 10 y 15 hilos, mientras que con 20 la TUF obtuvo el menor tiempo. Esto muestra que el equipo con mejor resultado cambia según la configuración y que un solo punto no describe toda la prueba.

Con 1,000 hilos, los tiempos fueron 1.139168 segundos en la MacBook, 0.558 en la Vivobook y 0.492 en la TUF. Con 2,000, la Vivobook obtuvo 0.526 segundos y superó los 0.600 de la TUF. Los dos equipos Windows lograron mejores mínimos que la MacBook, aunque con cantidades distintas de hilos.

La TUF obtuvo la mejor marca del benchmark y la MacBook mostró cambios más graduales entre configuraciones. La comparación corresponde a este programa y a los equipos utilizados, que tienen distintos procesadores, sistemas operativos y entornos de compilación.

13. Sobresuscripción de hilos

La sobresuscripción ocurre cuando se solicitan más hilos que los que el procesador puede ejecutar al mismo tiempo. En esta práctica, los mínimos se obtuvieron con 450, 2,000 y 1,000 hilos, aunque los equipos solo disponen de 8, 22 y 16 hilos lógicos, respectivamente.

Los hilos adicionales deben compartir los recursos del equipo. Por eso, agregar más también implica más trabajo para administrarlos y no garantiza reducir el tiempo. En las tres series, las cantidades mayores terminaron dando tiempos superiores a sus mejores valores.

Además, el programa combina e imprime una suma por cada hilo. Al aumentar NUM_THREADS también aumenta la cantidad de resultados que debe reunir y mostrar. Esto ayuda a entender que la prueba mide más que la división del cálculo entre hilos.

14. Consideraciones del benchmark

Las condiciones de ejecución también pueden influir en los tiempos. Aunque procuré cerrar la mayoría de las aplicaciones en mi MacBook, quedaron procesos en segundo plano; mis colegas pudieron tener otros procesos activos durante sus pruebas. Esto puede contribuir a las fluctuaciones observadas. También puede influir el sistema operativo y la forma en que administra los recursos cuando se le solicita trabajar con distintas cantidades de hilos. Son posibles explicaciones de las variaciones, sin que podamos asegurar que una sola sea la causa.

También deben distinguirse las mediciones de los espacios N/D. En mi MacBook no hay tiempos superiores a 2,000 hilos porque OpenMP no pudo asignar los recursos necesarios en el intento con 2,100; en la TUF faltan dos registros. Esos espacios no significan que la ejecución haya tardado cero segundos ni permiten comparar esas configuraciones.

15. Conclusiones

Esta práctica me permitió observar que utilizar más hilos no siempre hace que un programa termine más rápido. En los tres equipos hubo una mejora en parte de las pruebas, pero después los tiempos volvieron a aumentar. Por eso fue importante revisar toda la tabla y las gráficas, en lugar de considerar solo la mayor cantidad de hilos.

La TUF obtuvo el mejor tiempo con 0.492 segundos y 1,000 hilos; la Vivobook alcanzó 0.526 segundos con 2,000 y mi MacBook, 1.076189 segundos con 450. La MacBook mostró cambios más graduales, mientras que los equipos Windows tuvieron más variaciones al inicio y luego alcanzaron tiempos menores.

Lo que aprendí es que conviene probar diferentes cantidades de hilos y observar en qué punto dejan de ayudar. También comprobé que no todos los equipos responden igual y que pueden aparecer límites al intentar usar demasiados hilos, como ocurrió en mi MacBook. La actividad permitió relacionar lo visto en clase con resultados concretos en nuestras computadoras.